

Hardware Security (CMPE/ENEE 691)
University of Maryland Baltimore County

Fall 2017

Project

Due: Dec. 13, 2017

Final project of *Hardware Security* course deals with Concurrent Error Detection (CED) in crypto ciphers. In particular, AES cores are targeted, and the goal is to detect if any fault has been injected during the normal operation of the device.

Redundancy-based CED schemes are frequently used in crypto devices. In this specific project, you are to implement a CED scheme using time redundancy, i.e., all (or a subset of) operations are performed twice and the results of 1st and 2nd repeats are compared. If operations are only repeated in time, the faults that affect both computations (the original one and repeated version) may not be detected. To be able to detect such faults, re-computing with permuted schemes have been proposed in literature [1].

In this project, you are to design a time-redundant based scheme to detect bit-level and byte-level faults injected in any of AES main operations, including :s-box, shift-row, mix-column, and add-round-key.

Note1: You can use the AES implementation we had in lab and apply changes accordingly, or you can design your AES hardware. You can follow the method in [1] but preferably, you can come up with your invariances to be checked in your time-redundant fault detection scheme. Note that both permanent faults or transient faults that stay for 1 or more clock cycles should be detected.

Note2: Your design is an iterative design, i.e., you have only one sample of each of the following components: s-box, shift-row, mix-column, and add-round-key, and these components are used in different rounds iteratively. Therefore, you need to design a reconfigurable controller that is able to repeat each round for testing purposes. The fault should be reported (using an alarm signal) as soon as it is detected.

Note3: A two bit input signal (called *MODE*) determines if CED is going to occur. (“00”: Normal operation, “01”: repeating all rounds, “10”: repeating only one round which is determined by another 4 bit input called “*Round*”).

Note 4: You can use the fault injection mechanism we had in the lab but you need to change it to be able to inject permanent faults as well as transient fault that stay more than 1 clock cycle, and also support bit and byte level faults.

Deliverables:

1. A complete package including your circuit, scripts, Readme files, Makefiles, etc.
2. Demo: You should give a demo of the project using the testcases
3. Project report: Write an extensive report comparing the results obtained along with your observations. Use tables, graphs, etc to elaborate on your observations.

References:

- [1] X. Guo and R. Karri, "Re-computing with permuted operands: A concurrent error detection approach," IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems, 32(10), pp.1595-1608.